

Лабораторна робота №12

Тема

CI pipeline (GitHub Actions)

Мета

Налаштувати CI-пайплайн для статичного сайту, підключити автоматичну перевірку HTML і JavaScript, а також додати захист від потрапляння помилкового коду в main.

Що реалізовано

- Створено статичний сайт з файлами index.html, style.css, app.js.
- Налаштовано ESLint для перевірки JavaScript.
- Налаштовано HTMLHint для перевірки HTML.
- Додано GitHub Actions workflow у .github/workflows/ci.yml.
- Додано два jobs:
- Check Structure перевіряє наявність index.html, app.js, style.css.
- Lint Code запускає npm ci, npm run lint, npm run lint:html.

Локальна перевірка

```
npm ci
npm run lint
npm run lint:html
```

Демонстрація помилки для Pull Request

Для перевірки роботи CI можна створити гілку:

```
git checkout -b feature/broken-code
```

Потім навмисно додати в app.js помилку, наприклад невикористану змінну або == замість ===, зробити commit і push. У Pull Request job Lint Code має завершитись з помилкою, а merge буде заблоковано правилами захисту гілки.

Після виправлення помилки й повторного push CI має пройти успішно, а кнопка merge стане активною.

Місце для скріншотів

1. Pull Request з навмисною помилкою: червоний статус Lint Code, merge заблоковано.
2. Pull Request після виправлення: зелений статус Check Structure і Lint Code, merge доступний.

Скріншоти треба зробити після push у GitHub, бо локально GitHub Actions і branch protection не відображаються.

Відповіді на теоретичні питання

1. Чому в CI використовують npm ci замість npm install?

npm ci встановлює залежності строго за package-lock.json і не змінює lock-файл. Це важливо для CI, бо пайплайн має бути повторюваним: однаковий код і lock-файл повинні давати однакові залежності. npm install може оновити package-lock.json, тому він менш передбачуваний для автоматичних перевірок.

2. Що таке тригер у GitHub Actions?

Тригер - це подія, після якої GitHub запускає workflow. Наприклад, push доречний для перевірки кожного оновлення гілки, pull_request доречний перед злиттям змін у main, а schedule доречний для регулярних перевірок за розкладом.

3. Навіщо захищати гілку main правилами?

Домовленість у команді може бути випадково порушена. Branch protection робить правило технічним обмеженням: код не можна злити без Pull Request і успішних status checks. Це зменшує ризик потрапляння зламаного коду в головну гілку.

4. Що буде, якщо один step у job завершиться з помилкою?

Якщо step повертає ненульовий код виходу, GitHub Actions позначає його як failed. Зазвичай наступні steps у цьому job не виконуються, а весь job отримує статус failed.

5. Різниця між Continuous Integration і Continuous Deployment

У цій лабораторній реалізовано Continuous Integration: код автоматично перевіряється після push або Pull Request. Continuous Deployment ще не реалізовано, бо сайт не розгортається автоматично на хостинг після успішного проходження перевірок.